

Lab1 实验报告
陈宏彬 25303020005

C bits.c X

C bits.c

```
1  /*
2   * CS:APP Data Lab
3   *
4   * <Please put your name and userid here>
5   *
6   * bits.c - Source file with your solutions to the Lab.
7   *          This is the file you will hand in to your instructor.
8   *
9   * WARNING: Do not include the <stdio.h> header; it confuses the dlc
10  * compiler. You can still use printf for debugging without including
```

问题 输出 调试控制台 终端 端口

bash - datalab-1

```
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 10/24 operators
midpointTowardFirst: 19/32 operators
isBetweenEitherOrder: 30/48 operators
mul5Sat: 17/30 operators
classifyAdd3: 22/52 operators
floatScaleThreeHalves: 35/60 operators
floatRoundEven: 26/65 operators
float_i2f: 20/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
xr@LAPTOP-1C8MDIMM:/mnt/c/Users/xr/datalab-1$
```

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse
Total points: 110/110			
○ <code>xr@LAPTOP-1C8MDIMM:/mnt/c/Users/xr/datalab-1\$</code>			

各函数的实现思路

P1: signMask

将整数 1 左移 31 位，使原来最低位的 1 移到最高位，其余位均为零，从而得到要求的位模式 0x80000000。

P2: bitXor

异或在两个对应位不同时返回 1，因此分别用 $x \& \sim y$ 和 $\sim x \& y$ 表示两种不同的情况。由于本题不能直接使用按位或，利用德摩根律将两种情况合并，整个实现只使用 $\sim$ 和 $\&$ 。

P3: negativePart

使用 $\sim x + 1$ 得到取负后的位模式，再通过 $x \gg 31$ 构造符号掩码。负数的掩码为全一，保留取负结果；非负数的掩码为零，使结果为零。对于 `INT_MIN`，按实验规定的 32 位模型处理。

P4: copyByteWithin

先将源字节右移到最低位，并与 255 按位与，提取该字节。然后构造目标字节的掩码，清除原来的目标字节，再将源字节左移到目标位置，通过按位或写入，其他字节保持不变。

P5: logicalShift

先对 x 进行算术右移，再构造一个高 n 位为零、其余位为一的掩码。通过按位与清除算术右移可能补入的符号位，实现高位补零的逻辑右移。掩码的构造同时适用于 $n=0$ 和 $n=31$ 。

P6: swapNibblePairs

构造掩码 $0x0f0f0f0f$ ，提取每个字节的低四位并左移四位；同时将高四位右移，并用同一个掩码保留需要的部分。最后合并两部分，实现每个字节内部高、低半字节的交换。

P7: secondLowestZeroBit

先对输入取反，将查找零位转化为查找一位。利用 $v \& (v-1)$ 删除最低的一个一，再利用 $v \& (-v)$ 提取剩余的最低一，它对应原数的第二个最低零位。减一和取负均使用实验允许的补码运算表达；不足两个零位时返回零。

P8: oddParity

通过依次右移 16、8、4、2、1 位并与原值异或，将所有位的奇偶信息折叠到最低位。最低位为一表示原数有奇数个一，为零表示有偶数个一。本题要求补足奇校验，因此最后将该位取反：偶数个一返回 1，奇数个一返回 0。

P9: rotateRightBits

先用 $n \& 31$ 得到实际循环移位量。将结果分成逻辑右移部分和左移环绕部分，最后通过按位或合并。左移量也限制在 0–31 内，避免移位量为 32，并使移动 32 的倍数位时返回原值。

P10: roundEvenPow2

将 2^n 作为一个单位，右移得到原商。先给输入加上“半个单位减一”，再加上商的最低位，使超过中点的情况进位，而恰好位于中点时仅对奇数商进位。最后先右移再左移，得到最近的 2^n 的倍数，实现中点舍入到偶数商。

P11: midpointTowardFirst

利用数学恒等式 $x+y = 2 \times (x \& y) + (x \wedge y)$ ，通过 $(x \& y) + ((x \wedge y) >> 1)$ 得到向下取整的中点，避免直接相加溢出。当中点不是整数时，如果第一个参数较大，就给下界加一，使结果靠近第一个参数。比较大小时分别处理同号和异号情况，避免差值溢出导致错误。

P12: isBetweenEitherOrder

分别判断 $x < a$ 和 $x < b$ 。两个判断结果不同时，说明 x 位于两个端点之间；再单独接纳与任一端点相等的情况，构成包含端点的闭区间。大小比较区分同号与异号，因此能够正确处理负数和整数边界，也不依赖端点的输入顺序。

P13: mul5Sat

通过连续加法分别计算 $2x$ 、 $4x$ 和 $5x$ ，并检查每一步是否发生符号变化。由于这些步骤都是同号相加，符号变化可以用于判断溢出。若发生溢出，根据原输入符号选择 `INT_MAX` 或 `INT_MIN`；否则返回五倍结果。检查所有步骤可避免最终结果回绕后又与原数同号而漏判。

P14: classifyAdd3

将每个输入拆成右移两位得到的高部分和最低两位组成的低部分。先把低部分相加，再将产生的进位加到三个高部分的和中，这些中间计算不会超出 `int` 范围。根据最终高部分是否位于合法范围内，判断精确总和能否用有符号 32 位整数表示，分别返回 0、1 或 -1。

P15: floatScaleThreeHalves

先拆分浮点编码的符号、指数和小数字段，遇到 `NaN` 或无穷大时返回原编码。规格化数补上隐含的最高位一，将有效数字乘三，再通过右移完成除二和必要的规格化，并根据丢弃的位进行最近偶数舍入。最后处理非规格化数转为规格化数、舍入进位以及指数溢出，重新组合浮点编码，并保留符号。

P16: floatRoundEven

根据指数字段判断浮点数的小数部分所在位置。已经没有小数部分的值、`NaN` 和无穷大直接返回；绝对值小于或等于 0.5 的相应情况返回带原符号的零。其余

情况清除小数位，再比较丢弃部分与半程的大小，中点时根据保留整数的奇偶性决定是否进位，最后返回舍入结果的浮点编码。

P17: float_i2f

先记录符号，并用无符号整数保存绝对值，以处理 `INT_MIN`。通过不断左移找到最高有效位，同时确定浮点指数；保留 24 位有效数字，将其余位用于最近偶数舍入。最后合并符号、指数和小数字段，舍入造成有效数字进位时，相应调整指数。

P18: bitCount

采用分组计数方法：先统计每两位中的一的数量，再逐步合并成四位组和八位组的计数。随后通过右移和相加汇总各字节的计数，最终得到整个 32 位模式中一的数量。结果范围为 0–32，因此最后保留低六位即可。

P19: bitReverse

依次交换相邻位、两位组、四位组、八位组和十六位组，最终实现整个 32 位序列的反转。为了满足操作数上限，从十六位掩码开始，通过与自身左移结果异或，逐级构造其他掩码。最后一次右移使用掩码清除符号扩展，整个实现使用 34 次运算。